WEB PROGRAMMING

Day 1 — Foundations, Terminology & First HTML Programs

A beginner's reference sheet: what the words mean, why the code looks the way it does, and how a page finally becomes a live website.

1. The Big Picture — Three Languages, Two Sides

Before memorizing terms, it helps to see how the pieces fit together. Think of a webpage like a house:

- HTML (structure) — the walls, rooms, and doors. It decides what exists on the page: a heading here, a paragraph there, a button over there.
- CSS (style) — the paint, furniture, and lighting. It decides how things look: colors, fonts, spacing, layout.
- JavaScript (behavior) — the electricity and wiring. It makes things respond: a light turns on when you flip a switch, a menu opens when you click it.

Client-side vs. Server-side

Every website has two "sides" working together:

- Client-side = runs inside the user's own browser, on their device. HTML, CSS, and JavaScript are the classic client-side trio — the browser downloads them and does all the work locally.
- Server-side = runs on a remote computer (a server) before anything reaches the browser. It handles things like databases, logins, and business logic, using languages such as Node.js, Python, PHP, or Java, then sends the finished result to the client.

A simple way to remember it: client-side is what happens on your screen; server-side is what happens behind the scenes, on someone else's computer, before your screen even sees it.

---

2. Core Terminology

A. The Three Core Languages

Term	What it means
HTML	HyperText Markup Language — the markup language that defines a webpage's structure and content: headings, paragraphs, images, links, forms.
CSS	Cascading Style Sheets — the styling language that controls how HTML looks: colors, fonts, spacing, layout, and responsiveness.
JavaScript (JS)	The programming language that adds interactivity and logic to a page — responding to clicks, validating forms, updating content without reloading.
DOM	Document Object Model — the browser's live, in-memory tree version of your HTML page. JavaScript reads and changes the DOM to update what's on screen.
Tag	A keyword wrapped in angle brackets, e.g. <p> or <img>, that marks where an HTML element begins or ends.

Term	What it means
Element	A complete HTML unit: an opening tag, its content, and a closing tag — e.g. Hello is one element.
Attribute	Extra information placed inside an opening tag, written as name="value", e.g. lang="en" or src="photo.jpg".

B. Client-Side Terms

Term	What it means
Client	The user's device and browser — the one making requests and displaying the final webpage.
Browser	Software such as Chrome, Firefox, or Safari that requests, reads, and renders (draws) web pages for the user.
Frontend	Everything the user sees and interacts with directly — the HTML/CSS/JS running in the browser.
Rendering	The browser's process of turning HTML, CSS, and JS code into the visual page you actually see.
Responsive Design	Designing a page so its layout adapts smoothly to different screen sizes — phone, tablet, desktop.
Viewport	The visible area of a webpage on a given screen — what fits without scrolling.

C. Server-Side Terms

Term	What it means
Server	A remote, always-on computer that stores a website's files and data, and responds to requests from browsers.
Backend	The server-side part of an application — logic, databases, authentication — that users never see directly.
Database	Organized storage for data (e.g. MySQL, MongoDB, PostgreSQL) that the backend reads from and writes to.
API	Application Programming Interface — an agreed-upon way for two programs to talk to each other,

Term	What it means
	e.g. a frontend asking a backend for data.
HTTP / HTTPS	The protocol (rule set) browsers and servers use to exchange requests and responses. The "S" in HTTPS means the connection is encrypted and secure.
Request / Response	The browser sends a request ("give me this page"); the server sends back a response (the page or data).
Full-stack	Development work that covers both the frontend (client-side) and the backend (server-side) of an application.

D. Tools You'll Use Every Day

Term	What it means
IDE / Code Editor	Software for writing and organizing code, e.g. VS Code or Sublime Text. IDE stands for Integrated Development Environment.
Terminal / CLI	Command Line Interface — a text-based way to run commands on your computer instead of clicking icons.
Git	Version control software that tracks every change made to your code over time, so you can undo mistakes or see history.
GitHub	A website that hosts Git repositories online, so code can be backed up, shared, and worked on with others.
Repository ("repo")	A project's folder as tracked by Git — all its files plus their full history of changes.
npm	Node Package Manager — a tool that installs and manages reusable code packages for JavaScript projects.
Node.js	A runtime that lets JavaScript run outside the browser, e.g. directly on a server or your own computer.
localhost	A name meaning "this very computer" — used to preview a website on your own machine before it's published.
Port	A numbered "door" (e.g. :3000) on a computer that one running program listens on, so requests reach

Term	What it means
	the right app.

E. Hosting & Deployment — where cPanel and Vercel fit in

Term	What it means
Domain Name	The human-readable web address people type or click, e.g. abhishekshah.vercel.app, instead of a raw numeric address.
DNS	Domain Name System — the internet's "phonebook" that translates a domain name into the numeric address (IP) of the server that holds it.
Web Hosting	A service that stores a website's files on a server so the site is reachable on the internet at all times.
cPanel	A graphical control panel offered by traditional web hosting providers, used to manage files, databases, email accounts, and domains through a dashboard rather than the terminal.
Vercel	A modern hosting and deployment platform built especially for frontend frameworks like Next.js and React. It connects to a GitHub repository and automatically redeploys the site every time new code is pushed.
Netlify	A platform similar to Vercel, used for hosting and automatically deploying static and frontend web projects.
GitHub Pages	Free hosting for simple static sites, served directly out of a GitHub repository.
Deployment	The act of publishing your code so it becomes live and reachable on the internet.
Build	The step where raw source code is compiled, bundled, and optimized into the final files that actually get served to visitors.
Environment Variable	A configuration value (often a secret, like an API key) stored outside the code itself, so it isn't exposed in the source.

F. Other Common Terms

Term	What it means
Framework	A structured toolkit that provides rules and pre-built building blocks for an app, e.g. React, Django, Laravel — you build within its structure.
Library	A collection of reusable code you call whenever you need it, e.g. jQuery — it gives less imposed structure than a framework.
Compiler / Interpreter	Software that translates the code you write into instructions a computer can execute. JavaScript is interpreted — read and run line by line — by the browser's JS engine.
Runtime	The environment where code actually executes. A browser is JavaScript's runtime on a webpage; Node.js is a runtime for running JS outside the browser.
SEO	Search Engine Optimization — practices that help a webpage rank higher and appear more clearly in search engine results.
Favicon	The small icon shown in a browser tab (and bookmarks) representing a website.
Semantic HTML	Using tags for their true meaning — e.g. <header>, <nav>, <footer> — instead of generic s everywhere, which helps accessibility tools and search engines understand the page.

---

3. Your First HTML Programs — Explained Word by Word

Below are five short programs, each building on the last. Every new tag and attribute is explained immediately after its code block — including things like why the very first line exists, and what lang="en" actually does.

Program 1 — The Minimal Skeleton

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, World!</h1>
</body>
</html>
```

Code / Word	Meaning
`<!DOCTYPE html>`	Not a tag — a declaration at the very top of every HTML file. It tells the browser "read this as modern HTML5," so the browser uses standard, predictable rendering rules instead of falling back to old, inconsistent "quirks mode" behavior from the 1990s web.
`<html lang="en">`	The root element — everything else on the page lives inside it. The lang="en" attribute declares the page's language as English. Screen readers use it to choose the right pronunciation, browsers use it for translation prompts, and search engines use it to serve the page to the right audience.
`<head> ... </head>`	A container for information about the page that is not itself shown on screen — things like the title, character encoding, and links to CSS/JS files. Think of it as the page's ID card.
`<meta charset="UTF-8">`	Sets the character encoding to UTF-8, the standard that supports virtually every character and symbol (emoji, accented letters, non-English scripts) without displaying as garbled text.
`<title>My First Page</title>`	The text shown in the browser tab, and used by default as the bookmark name and search-engine result title.
`<body> ... </body>`	Contains everything that is actually visible on the page — text, images, buttons, everything a visitor sees.
`<h1>Hello, World!</h1>`	A level-1 heading — the largest, most important heading on the page. HTML supports headings from <h1> (biggest) down to <h6> (smallest).

Program 2 — Text, Links, Images, and Lists

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>About Me</title>
</head>
<body>
  <!-- This is a comment; the browser ignores it -->
  <h1>About Me</h1>
  <p>I am learning web programming, one page at a time.</p>

  <img src="profile.jpg" alt="My profile photo">
```

```
  <a href="https://github.com/abhiverse01" target="_blank">Visit my GitHub</a>

  <h2>My Interests</h2>
  <ul>
    <li>Machine Learning</li>
    <li>Full-stack Development</li>
    <li>Open Source</li>
  </ul>
</body>
</html>
```

Code / Word	Meaning
`<!-- ... -->`	An HTML comment. Anything between <!-- and --> is ignored by the browser — used for notes to yourself or other developers.
`<p>...</p>`	A paragraph of regular text.
`<img src="..." alt="...">`	Embeds an image. src (source) points to the image file's location. alt is a text description shown if the image fails to load, and read aloud by screen readers — required for accessibility. Note: <img> has no closing tag; it's self-contained.
`<a href="..." target="_blank">`	An anchor (link). href (hypertext reference) is the destination URL. target="_blank" tells the browser to open that link in a new tab instead of replacing the current page.
`<h2>...</h2>`	A level-2 heading — one step smaller/less important than <h1>, typically used for section titles.
`<ul>...</ul>`	An unordered list — a bulleted list, with no numbering.
`<li>...</li>`	A list item — one entry inside a (or its numbered cousin,).

Program 3 — A Simple Table

```
<table>
  <tr>
    <th>Language</th>
    <th>Purpose</th>
  </tr>
  <tr>
    <td>HTML</td>
    <td>Structure</td>
  </tr>
  <tr>
    <td>CSS</td>
    <td>Style</td>
  </tr>
</table>
```

Code / Word	Meaning
`<table>...</table>`	Defines a data table made of rows and columns.
`<tr>...</tr>`	Table Row — one horizontal row of the table.
`<th>...</th>`	Table Header cell — a heading cell, usually shown bold and centered by default.
`<td>...</td>`	Table Data cell — a normal cell holding one piece of data.

Program 4 — A Simple Form

```
<form action="/submit" method="POST">
  <label for="name">Your Name:</label>
  <input type="text" id="name" name="name" placeholder="Enter your name">

  <label for="email">Your Email:</label>
  <input type="email" id="email" name="email" placeholder="you@example.com">

  <button type="submit">Send</button>
</form>
```

Code / Word	Meaning
`<form action="..." method="POST">`	A container for user input. action is the URL the data is sent to (often a server-side endpoint). method describes how it's sent — POST sends data behind the scenes; GET appends it visibly to the URL.
`<label for="name">`	Text describing an input field. The for attribute must match the input's id, linking label to field so clicking the label focuses the input — important for accessibility.
`<input type="text" id="name" name="name" placeholder="...">`	A single-line text field. type sets the kind of input (text, email, password, etc.). id uniquely identifies this element on the page (used by labels and CSS/JS). name is the key sent to the server when the form is submitted. placeholder is greyed-out hint text shown when the field is empty.
`<input type="email" ...>`	Same idea as above, but type="email" makes the browser lightly validate that the entry looks like an email address before submitting.
`<button type="submit">`	A clickable button. type="submit" makes it submit the form's data when clicked.

Program 5 — Bringing HTML, CSS, and JavaScript Together

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Interactive Page</title>
  <style>
    body {
      font-family: sans-serif;
      text-align: center;
    }
    .highlight {
      color: white;
      background-color: #1B2A4A;
      padding: 10px;
    }
  </style>
</head>
<body>
  <h1 id="greeting">Hello!</h1>
  <button id="myButton">Click Me</button>

  <script>
    const button = document.getElementById("myButton");
    const heading = document.getElementById("greeting");

    button.addEventListener("click", function () {
      heading.classList.add("highlight");
      heading.textContent = "You clicked the button!";
    });
  </script>
</body>
</html>
```

Code / Word	Meaning
`<style>...</style>`	Holds internal CSS — rules that apply only to this page. Placed inside <head>.
`body { font-family: sans-serif; }`	A CSS rule. body is the selector (what to style). Inside the curly braces, each line is a property: value; pair — here, font-family sets the typeface.
`.highlight { ... }`	A CSS class selector — the dot means "any element with class=\"highlight\"." Classes let you reuse the same style on multiple elements, unlike an id, which must be unique to one element.
`<h1 id="greeting">`	The id attribute gives this specific element a unique name ("greeting") so CSS or JavaScript can target it directly.
`<script>...</script>`	Holds JavaScript code. Placed near the end of <body> so the HTML above it loads first, before the script tries to interact with it.

`const button = document.getElementById("myButton");`	document is the DOM (the page itself, as JS sees it). getElementById finds the one element whose id matches, and const stores a reference to it in a variable named button.
`button.addEventListener("click", function () {...});`	Attaches a listener that waits for a "click" event on that button, then runs the function inside the curly braces whenever it's clicked.
`heading.classList.add("highlight");`	Adds the CSS class "highlight" to the heading element at that moment, instantly applying the styling defined for .highlight.
`heading.textContent = "...";`	Replaces the heading's visible text with a new string, updating the page without a reload — this is the core trick behind interactive web pages.

---

4. What to Do Next

- Save Program 1 as index.html and open it directly in a browser — no server needed yet.
- Change the text, add another <p>, add a second image — small edits build intuition fastest.
- Once comfortable, move Program 5 into three separate files (index.html, style.css, script.js) and link them — this is the real-world structure almost every project uses.
- When ready to publish, push the project to a GitHub repository, then connect it to Vercel or Netlify for a free, instant, auto-updating live link.

Good luck with Day 1 — the fundamentals here carry through everything else in web development.